# Generalized Linear Models and Distributions

BIOL 3401 Mount Royal University

Topic 7, Winter 2026

## Contents

**Learning Objectives** **1**

Introduction to modelling in R . . . . . 1

Set up your R workspace for the day . . . . . 2

Part 1: Logistic regression . . . . . 2

Part 2: Poisson regression . . . . . 6

Part 3: Translating effects from link functions . . . . . 7

Part 4: Plotting non-linear predictions . . . . . 8

Part 5: Overdispersion . . . . . 10

## Learning Objectives

Today, we will start learning about statistical models. Linear models are one of the simplest and most versatile classes of statistical model. We will add more complexity to our analyses in the weeks to come.

After completing this topic, you should feel comfortable:

1. Comparing and contrasting Generalized Linear Models (GLMs) and simple linear models.
2. Identifying appropriate link functions and distributions (e.g., binomial, Poisson) for different types of response variables.
3. Fitting GLMs and interpreting results based on the link function.
4. Evaluating GLM model fit.

### Introduction to modelling in R

In many ways, GLMs are more straightforward and more versatile than simple linear models because their assumptions are not quite as restrictive. Instead of checking whether the residuals are normally distributed and independent with homogeneous variance, and ensuring the Y ~ X relationship is linear, we can often just use our understanding of how the data were generated to make the correct modelling choices.

## Set up your R workspace for the day

### Set your working directory

Make sure we're setting our working directory!

```
setwd()
```

### Install and load packages

We are going to need to load `tidyverse` and `car`. We are also going to install two new packages called `DHARMa` and `MASS`.

Using the code block below, install the two new packages and load all four packages.

### Load data

We have two datasets for this topic.

- **ivf_data.csv**: Comes from a retrospective study conducted at a fertility clinic. The researchers evaluated predictors of clinical pregnancy (age and hormone treatment) following a single in-vitro fertilization (IVF) cycle. Each row represents a unique patient.
  - `pregnant`: Whether the patient achieved a clinical pregnancy (1 = yes, 0 = no)
  - `age`: The age of the patient at the time of egg retrieval (units: years)
  - `dose`: The total daily dose of recombinant follicle-stimulating hormone (rFSH) administered during ovarian stimulation (IU/day)
  - `prev_failure`: Whether the patient had at least one previous failed IVF cycle (1 = yes, 0 = no)
- **pollen_data.csv**: Comes from a field experiment examining how nitrogen enrichment and plant biomass influence grassland pollinator abundance. Plots were surveyed between July 15th and 30th, 2006 when pollinator visits were counted. Each row represents a unique plot.
  - `visits`: Number of visits from pollinators of all species recorded during the 15-minute observation window.
  - `fertilized`: Whether the plot received 10 g/m^2 nitrogen fertilizer once at the start of the season (1 = yes, 0 = no)
  - `biomass`: Aboveground plant biomass harvested at peak season (g/m^2)
  - `area`: Precise plot size surveyed, accounting for slight variations (m^2)

Load the datasets by finishing the `read_csv()` function.

```
pollen_data <- read_csv()
ivf_data <- read_csv()
```

## Part 1: Logistic regression

Logistic regression tells us how the probability that X is a success in N trials, given a probability of success = p, changes as a function of some independent or explanatory variable(s).

Using our IVF data, we are going to test the hypothesis that *aging reduces the efficacy of IVF treatment.* Our prediction is that the probability of achieving clinical pregnancy declines with age.

### Part 1a: Fit the models

We are going to jump right into model-fitting to test our hypothesis. We will use the familiar function `lm()` and a new function, `glm()`, which fits generalized linear models. `glm()` takes the same arguments as `lm()`: the model formula (`y ~ x`) and `data =`. It also has an additional argument: `family =`. This argument specifies the details of how the function should fit the model. Among others the function recognizes the following family arguments corresponding to models we've already talked about:

- Linear regression: `gaussian(link = "identity")`
- Logistic regression: `binomial(link = "logit")`
- Gamma regression: `Gamma(link = "log")`
- Poisson regression: `poisson(link = "log")`

Note that the family specification *should not be in quotes.* For now, we will avoid talking about the `(link = )` components of the arguments.

You likely recognize the term "Gaussian", which refers to our familiar normal distribution. Simple linear models are just a special version of GLMs! We could also fit a simple linear model using the Gaussian family and `glm()`:

```
# Simple linear model using lm()
lm(y ~ x, data = dat)

# Same model fit using glm()
glm(y ~ x, data = dat, family = gaussian(link = "identity"))
```

We know that we want to test the effect of age on pregnancy. We also know that the outcome, "pregnant", is binary (yes/no), so we're looking at logistic regression.

### Mini Challenge 1

1. Fit a simple linear model to test the effect of `age` on `pregant`, using the `ivf_data`.
2. Fit a logistic regression model to test the effect of `age` on `pregant`, using the `ivf_data`, and specifying the appropriate family argument.
3. Call the models `pregnancy_lm` and `pregnancy_glm`

Use the code block below for your answer!

#————————————————————————-

### Part 1b: Assess model fit

Now, we'll take a look at the model summaries, assessing goodness of fit and effect sizes.

```
# Summarize the simple linear regression
summary(pregnancy_lm)

# Summarize the logistic regression
summary(pregnancy_glm)
```

Both models seem OK from their summaries. Both effect sizes for age seem to show that achieving pregnancy or the probability declines with age. The intercept in this case doesn't make much biological sense, as it tells us about pregnancy when age = 0, an impossibility.

Note that the `glm()` model *does not have an R-squared in the summary.* **(Why?)**

### Part 1c: Check model assumptions

Next, we'll use `plot()` to check assumptions from the simple linear regression.

```
plot(pregnancy_lm)
```

Looks like some major violations!

What about the logistic regression model? We can generate Q-Q and scatter plots of our residuals with the `simulateResiduals()` function from the `DHARMa` package, which generates the residuals from our model without assuming a distribution. The function takes the model as an argument `fittedModel =`. We can also specify the argument `plot = TRUE`, which generates the plots for our test.

```
simulateResiduals(fittedModel = pregnancy_glm, plot = TRUE)
```

What we are seeing here is the result of a Kolmogorov-Smirnov test (KS test) on our QQ plot. The Kolmogorov-Smirnov is essentially a non-parametric equivalent of a Shapiro-Wilk test, but instead of testing whether your residuals are approximately normal, you are testing if your residuals are consistent with the distribution you chose when you specified the family.

If this is basically a nonparametric Shapiro-Wilks test, so a non-siginficant result suggests we chose a good family (distribution) for our response variable.

You also see results from an outlier test. We interpret this p-value in the same way as Cook's distance: non-significant results mean no outliers.

The other important thing to note is our "overdispersion test". We'll come back to the idea of overdispersion later, but for now you should note we interpret it in the same way as other tests (non-significant result = no overdispersion).

### Part 1d: Plot the model predictions

It will be easier to see why the simple linear model violates our assumptions so much if we plot what the model predicts about the relationship between age and achieving pregnancy.

### Mini Challenge 2

Fit a scatterplot of the relationship between age and achieving pregnancy.

Use the `ivf_data` to create a scatterplot of `age` (dependent/explanatory variable) and `pregnant` (dependent/response variable)

Use the code block below for your answer!

#----------

The plot may appear strange at first, but it does represent the data well. The only options for `pregnant` are 1 or 0. There are many ages where patients *do not* achieve pregnancy (0), but also many at which they do (1). We can see that many of the unsuccessful pregnancies occur at the older ages, and successful pregnancies tend to be younger.

What does our model say about the relationship? We can find out by making predictions from our model. The `predict()` function requires the arguments `object =` and `newdata =`. The *"object"* is the model we want to predict from. *"newdata"* is a data.frame or tibble, which must contain a sequence of values with *the same name* as the independendent/explanatory variable. R will use this sequence to generate a sequence of predictions for the dependent/response variable. We can make any length of sequence with any values we want, as long as they are values we could conceivably see. `se.fit =` is a optional logical argument specifying whether we want to predict the error in the response variable as well as the estimate (*TRUE* means yes, include the error).

### Mini Challenge 3

Make predictions from the `pregnancy_lm` model.

1. Create a sequence of ages (independent variable) from the minimum age in `ivf_data` to the maximum, increasing by increments of 0.1.
2. Make the sequence a column "age" in a new tibble called "predict_preg_data".
3. Use `predict()` to predict from `pregnancy_lm`, specifiying se.fit = TRUE.
4. Call the predictions "predict_preg_lm"

Use the code block below for your answer!

The output from our predictions is a list object. The first two elements of the list are vectors:

- **fit**: The mean predictions, the same length as the vector we provided
- **se.fit**: The error around the prediction

We also have values with some other information from the model.

We'll add the predictions (and error) to `predict_preg_data` using the `cbind()` function.

```
predict_preg_data <- cbind(predict_preg_data, predict_preg_lm[1:2])
```

We're going to add two new elements to our scatterplot: a line and a "ribbon", which gives us the error around the mean prediction from the model. We know that `ggplot2` already plots the mean model predictions and error with the `geom_smooth(method = "lm")` function. However, when we start to introduce more complex relationships with `glm()`, the "lm" method will no longer be appropriate. We have to predict and plot the relationship manually.

The `geom_ribbon()` needs a `ymin =` and `ymax =` in the aesthetic function to specify the boundaries of the ribbon. The boundaries are fit - se.fit (lower) and fit + se.fit (upper). I've also specified "alpha = 0.5" so the ribbon is transparent and we can see the points behind it.

```
ggplot() + geom_point(data = ivf_data, aes(x = age, y = pregnant)) +
    geom_ribbon(data = predict_preg_data, aes(x = age, ymin = fit -
        se.fit, ymax = fit + se.fit), alpha = 0.5) + geom_line(data = predict_preg_data,
    aes(x = age, y = fit))
```

This new plot makes it clear why our model didn't fit well. Clearly there is a negative relationship, but it is not well specified by a linear relationship.

### Part 1e: Compare model fits

We don't get an R-squared from our `pregnancy_glm` because this assessment of goodness of model fit doesn't make sense for logistic regression. However, we can still compare model fits using AIC.

### Mini Challenge 4

Compare AIC between `pregnancy_lm` and `pregnancy_glm`.

Use the code block below for your answer!

Which model provides a better fit to the data?

#————————————————————-

## Part 2: Poisson regression

Poisson regression tells us how the rate of an event changes as a function of some independent or explanatory variable(s).

Using our pollinator data, we are going to test the hypothesis that *pollinators are attracted to areas of greater plant biomass*. Our prediction is that the rate of pollinator visits increases with plant biomass.

This is a case for Poisson regression because we're interested in how the number of pollinator visits (count data) changes with biomass. We'll follow the same steps we did for logistic regression, starting with:

1. Fitting the model,
2. Assessing the fit,
3. Checking assumptions, and
4. Comparing the models using AIC.

We'll leave plotting our predictions for later.

### Part 2a: Fit the models

First, we'll fit a simple linear model and generalized linear model.

#### Mini Challenge 5

1. Fit a simple linear model to test the effect of `biomass` on `visits`, using the `pollen_data`.
2. Fit a poisson regression model to test the effect of `biomass` on `visits`, using the `pollen_data`, and specifying the appropriate family argument.
3. Call the models `visits_lm` and `visits_glm`

Use the code block below for your answer!

#———————————————————-

### Part 2b: Assess model fit

Let's take a look at the models:

```
# Summarize the simple linear regression
summary(visits_lm)

# Summarize the logistic regression
summary(visits_glm)
```

Again, both models seem to suggest a similar positive relationship between biomass and visits.

### Part 2c: Check model assumptions

Next, we'll use `plot()` and `simulateResiduals()` to check our assumptions.

#### Mini Challenge 6

Visually check the assumptions for both models, using the appropriate approach for each model type.

Use the code block below for your answer!

#————————————————————-

The diagnostics for our Poisson regression look OK, but our p-value from our dispersion test is getting close to significant. We also see that the residuals do not follow our predictions quite as well as before. We will come back to this. For now, this model looks a lot better than the simple linear regression!

#### Part 2d: Compare models with AIC

The Poisson regression looks better, but we have some concerns. Let's compare between models to see if the glm is better.

#### Mini Challenge 7

Compare your models using AIC.

Use the code block below for your answer!

#————————————————————-

## Part 3: Translating effects from link functions

We know that specifying a link functions is preferable to transforming the response variable, but it also means our response is *not* normally distributed around our predictions. Link functions "translate" the relationship between the values from our explanatory/independent variable and the mean of the response/dependent variable. Because of this, the effect sizes from our models don't necessarily have the same interpretation as a simple linear model.

Let's take a look at our `family` functions again:

- Linear regression: `gaussian(link = "identity")`
- Logistic regression: `binomial(link = "logit")`
- Gamma regression: `Gamma(link = "log")`
- Poisson regression: `poisson(link = "log")`

You can see that each function includes a specified *link*. This *link* is the link function default when you specify that family.

We already fit models using the Poisson and Binomial link functions, so we will examine the effect sizes from those models more closely.

#### Part 3a: Logit link interpretation

Let's look at the summary of our `pregnancy_glm` model.

```
summary(pregnancy_glm)
```

The "Estimate" from our model is -0.24. In simple linear regression, using the *"identity"* link, this would suggest that pregnancy decreases by -0.24 for every unit increase in age. However, this interpretation doesn't exactly make sense in logistic regression where we are using the *"logit"* link. Here, the effect size is *the change in the log-odds of success for each one-unit increase in the predictor*.

In our example, each one unit increase in age decreases the *log-odds* of achieving pregnancy by 0.24.

Log-odds aren't the most intuitive. If we want a more real-world interpretation, we can convert the log-odds to an *odds ratio*. The *odds ratio* gives us the *increase or decrease in the odds of success for every unit increase in the predictor*. To convert to the odds ratio, we exponentiate the effect size using the `exp()` function. This function just performs math, so the only argument is our effect size.

```
exp(-0.2374)
```

```
## [1] 0.7886758
```

The odds ratio is ~ 0.79. *This means that each unit increase in age decreases the odds of achieving pregnancy by approximately 21%* (1 - odds ratio).

Odds ratios < 1 are interpreted as a decrease in the odds. Ratios > 1 mean that the odds have increased. For example, if our effect size was + 0.2374, the odds ratio would be ~ 1.27, which would mean an approximate 27% *increase* in the odds of achieving pregnancy.

### Part 3b: Log link interpretation

Poisson regression generally uses the *"log"* link. The *log* link adds a multiplicative effect. Unlike the *logit* link, where we had to exponentiate the effect size to get the odds ratio, the Poisson effect size is essentially already an odds ratio. *We interpret it just like the odds ratio after exponentiating our effect size from the logistic regression model.*

Let's look at the summary from our pollinator visits model again.

```
summary(visits_glm)
```

The effect size from our model is ~ 0.01. This means that there is an approximate 1% increase in the odds of pollinator visits for each unit increase in biomass.

The interpretation is the same for other models that use the log-link function (i.e., Gamma and log-linear).

## Part 4: Plotting non-linear predictions

We already saw that we could plot the predictions from our simple linear regression models using the `predict()` function. Plotting the predictions from our generalized linear regressions will help us visualize the non-linear (log/logit) effect sizes.

### Part 4a: Plot predictions from logistic regression

We will start by visualizing the predictions from our `pregnancy_glm` model.

First, we'll predict the log-odds from our generalized linear model across a range of ages.

We need one more argument in the `predict()` function we didn't need to include for our simple linear regression. Because this is a GLM, we need to specify that we want to predict the *"response"*, rather than the *"link"*, which would give us the translation of our response variable into a line. We can specify what we want using the `type =` argument.

```
# Tibble with ages
predict_preg_glm_data <- tibble(age = seq(min(ivf_data$age),
    max(ivf_data$age), by = 0.1))
# Predict the response for each age in the tibble
predict_preg_glm <- predict(pregnancy_glm, newdata = predict_preg_glm_data,
    type = "response", se.fit = TRUE)
```

Next, we will add the predictions and standard error to our `predict_preg_glm_data` tibble.

```
predict_preg_glm_data <- cbind(predict_preg_glm_data, predict_preg_glm[1:2])
```

Finally, let's plot the results, overlaying our scatterplot.

```
ggplot() + geom_point(data = ivf_data, aes(x = age, y = pregnant)) +
    geom_ribbon(data = predict_preg_glm_data, aes(x = age, ymin = fit -
        se.fit, ymax = fit + se.fit), alpha = 0.5) + geom_line(data = predict_preg_glm_data,
    aes(x = age, y = fit))
```

We can see how this plot better represents the nature of the odds ratio as the probability of pregnancy changes with age.

### Part 4b: Plot predictions from Poisson regression

We know from our interpretation of the effect sizes that the relationship between biomass and pollinator visitation is also nonlinear in our `visits_glm` model.

### Mini Challenge 8

Plot the predictions from the `visits_glm` model.

Use the code blocks below for your answer!

Step 1:

- Create a tibble with a sequence of biomass from the minimum to maximium in `pollen_data`, increasing by increments of 0.5.
- Predict the response from the `visits_glm` model.

Step 2:

Add the predictions and standard error to the tibble with the biomass sequence.

Step 3:

Plot the relationship.

#———————————————————————-

## Part 5: Overdispersion

We need to revisit something we touched on earlier when we checked our GLM assumptions using `simulateResiduals()`. We saw that although the Poisson model fit a lot better than the linear model, our "dispersion test" was nearly significant, which suggests our response variable might be "overdispersed" for our Poisson regression model.

Overdispersion is very common with count data. One of the major properties of the Poisson distribution is that the mean of the distribution must equal the variance (rate parameter, $\lambda$). However, this is rarely the case for real data, where we often see many smaller counts and a long right tail with a few very large counts. Very often, the variance is larger than the mean.

### Part 5a: Assessing overdispersion

Let's simulate a Poisson-distributed vector and plot a histogram to show you what this distribution looks like when the variance equals the mean. We are using the `rpois()` function, which requires the arguments `n =` (number of values) and `lambda =` (the rate parameter).

```
# Simulate the data
sim_poisson <- rpois(n = 80, lambda = 3)

# Plot a histogram
hist(sim_poisson)
```

Now let's look at the distribution of our `visits` variable from `pollen_data`.

```
hist(pollen_data$visits)
```

The data in the simulated distribution has a roughly similar range of counts, but the mean is closer to the variance. Let's take a look.

```
# Mean and variance from our simulated Poisson data
mean(sim_poisson)
var(sim_poisson)
# Mean and variance from our visit data
mean(pollen_data$visits)
var(pollen_data$visits)
```

The visit data mean is almost twice as large as the variance! This is creating mild overdispersion in our data.

The `DHARMa` package has a useful function `testDispersion()` for testing for overdispersion. The argument required by this function is `simulationOutput`, which is our output from the `simulatedResiduals()` function. The test will provide the p-value from the plot we used to assess our model assumptions. It also shows us what the distribution of residuals should look like if the data matched the assumptions of our model.

```
sim_resids <- simulateResiduals(visits_glm)
testDispersion(sim_resids)
```

### Part 5b: Negative binomial regression

A natural place to start when we think we might have overdispersed count data is to use "negative binomial" models. The negative binomial regression adds an additional parameter, $\theta$, which accounts for the increased variance relative to the mean.

We need a new family function, `negative.binomial()`, which comes from the `MASS` package. We will fit a new negative binomial model, trying first with theta = 8. Note that you might have to adjust your theta to find the right fit!

```
# Fit a negative binomial model
visits_glm_nb <- glm(visits ~ biomass, data = pollen_data, family = negative.binomial(theta = 8,
    link = "log"))
# Test for overdispersion
sim_resids_nb <- simulateResiduals(visits_glm_nb)
testDispersion(sim_resids_nb)
```

Much better!